PROYECTO MLOPS · DEFENSA ORAL

Predecir pedidos atrasados, de MLflow a Kubernetes

Seguimiento con MLflow, contenedor Docker, 3 réplicas en Kubernetes y detección de drift

Karem Huacota Saavedra
Yesika Luna Robles

PROBLEMA

Avisar de un atraso antes de que ocurra

Qué predice

Si un pedido llegará tarde al cliente (late = 1), en el momento de despacharlo.

Con qué datos

5.000 pedidos sintéticos y 8 variables: distancia, peso, artículos, hora, carga del almacén, clima, operador y tipo de envío.

Para qué sirve

Avisar al cliente o reasignar recursos. Entre el 20 y el 25 % de los pedidos se atrasa.

El foco no es la exactitud del modelo, sino la ingeniería que lo rodea: trazabilidad, despliegue y monitoreo.

ARQUITECTURA

El modelo viaja de MLflow a 3 pods

1 · Datos sintéticos
generate_data.py, 5.000 pedidos y 8 variables

→

2 · Entrenamiento
train.py, 6 runs comparados por ROC-AUC

→

3 · MLflow
Registry: versión 1 con alias champion

4 · Imagen Docker
late-delivery-api:1.1, con el modelo dentro

→

5 · Kubernetes
3 pods y Service NodePort 30080

→

6 · Interfaz Streamlit
Consume el NodePort 30080

El monitoreo de drift lee el modelo exportado y data/reference.csv, sin tocar el servicio.

FASE 1 · MLFLOW

Cada modelo desplegado se rastrea hasta su run

Elemento	Valor
Experimento	late-delivery
Runs comparados	6, RandomForest
Métrica principal	ROC-AUC
Modelo registrado	late-delivery-classifier
Versiones	2
Desplegada	versión 1, alias champion
Run de origen	a2eb441b26c245c9bee71704f12e858d

Demo en vivo

- Ordenar los runs por roc_auc
- Comparar 2 o más runs
- Abrir el Model Registry
- Seguir el run_id de la versión 1

FASE 2 · DOCKER

La imagen es autosuficiente y reproducible

Dockerfile propio

Parte de python:3.11-slim, la misma versión de Python con la que se entrenó.

Dependencias fijadas

mlflow-skinny 2.16.2, scikit-learn 1.5.2, pandas 2.2.3, numpy 1.26.4 y FastAPI 0.115.0.

Modelo en la imagen

Opción A: los pods no dependen de MLflow. La API devuelve el pod, la versión y el run.

Costo de la decisión: cambiar de champion exige exportar el modelo y construir una imagen nueva.

FASE 3 · KUBERNETES

Un Deployment mantiene 3 réplicas detrás de un Service

Elemento	Configuración
Deployment	3 réplicas iguales
Imagen	late-delivery-api:1.1
readinessProbe	GET /health, controla el tráfico
livenessProbe	GET /health, reinicia si se cuelga
Service	NodePort 30080 hacia el puerto 8000
Recursos	100m CPU y 256Mi; límite 768Mi

3

réplicas en Running

localhost:30080

DEMOS EN VIVO

Cuatro demostraciones sobre el clúster

1

Réplicas en Running

kubectl get pods: 3 pods y READY 3/3.

2

Balanceo de carga

30 peticiones al Service, atendidas por pods distintos.

3

Autorreparación

Se borra un pod, nace otro y el servicio sigue respondiendo.

4

Escalado

Se escala a 5, a 2 y de vuelta a 3 réplicas.

El reparto es aleatorio por conexión, por eso no sale exactamente igual en cada pod.

DRIFT · ENTRADAS

Data drift: cada variable contra el baseline

Tipo	Variables	Prueba
Numérica	6 variables	Kolmogorov-Smirnov
Categórica	carrier, shipping_type	Chi-cuadrado
Umbral	p-value < 0.00125	0.01 / 8 pruebas

El umbral usa la corrección de Bonferroni. Con la semilla 7, el lote normal dio una falsa alarma esperada de cerca del 1 %.

VERDE · sin drift

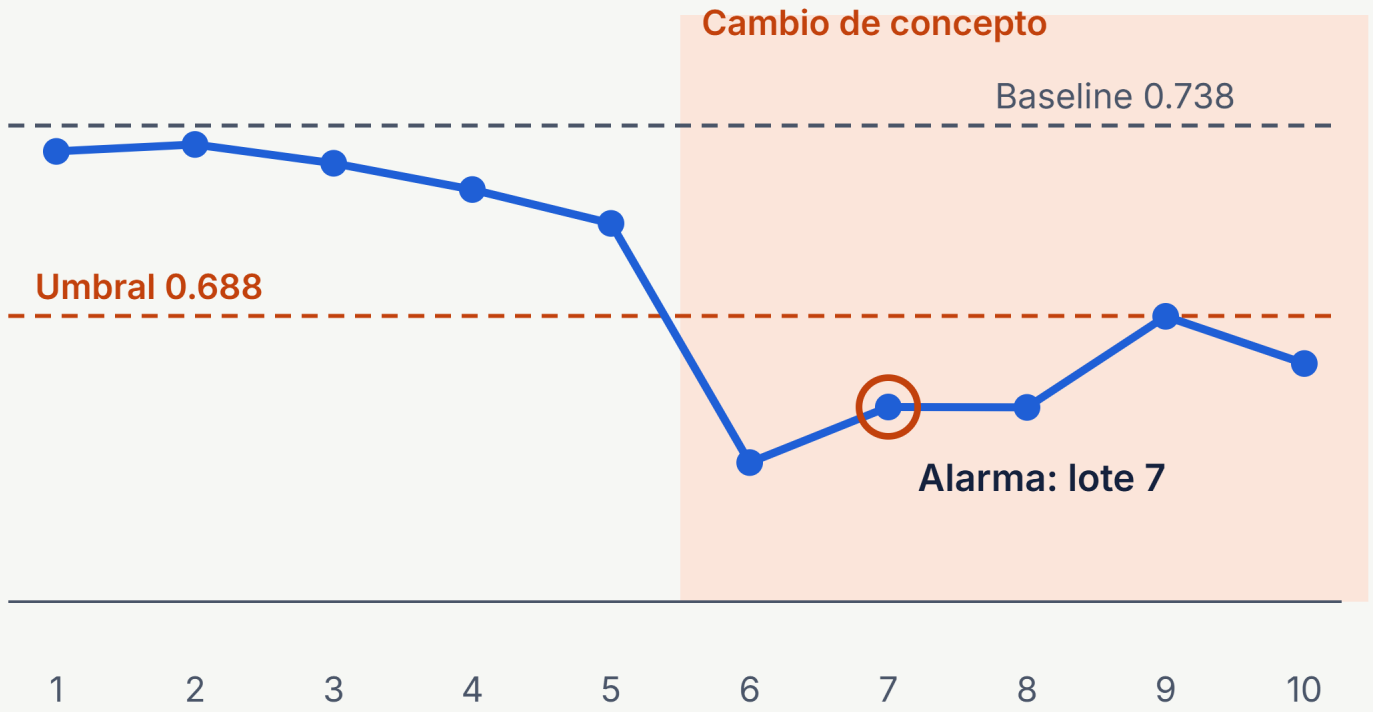
Lote normal de 1.000 pedidos.

ROJO · drift detectado

Temporada alta: más distancia, peso y carga.

DRIFT · CONCEPTO

Concept drift detectado en el lote 7



ROC-AUC por lote de 1.000 pedidos

Criterio

Caída de más de 0.05 durante 2 lotes seguidos.

Lote 6 · 0.6495

Primer lote bajo el umbral.

Lote 7 · 0.6641

Segundo lote seguido: se dispara la alarma.

DRIFT · RETRASO DE ETIQUETAS

Etiquetas tardías: qué hace el equipo

1 · Señales sin etiqueta

Data drift de las entradas y distribución de las predicciones.

2 · Evaluar con retraso

Guardar las predicciones y calcular el ROC-AUC cuando llegan las etiquetas.

3 · Etiquetas parciales

Los atrasos graves se conocen antes y sirven de muestra rápida.

4 · Respuesta provisional

Avisar al negocio, revisar una muestra y preparar el reentrenamiento.

En la simulación, la probabilidad media predicha se mantuvo cerca de 0.24 tras el cambio: solo el ROC-AUC, que necesita la etiqueta, lo delató.

EXTRA · INTERFAZ

La interfaz consume el clúster

Formulario

Envía los 8 campos a POST /predict y muestra el pod.

Barra lateral

Versión del modelo y run que sirve el clúster.

Demo de balanceo

Gráfico de peticiones atendidas por cada pod.

Prueba: el pod que muestra la interfaz coincide con kubectl get pods.

SERVICIO CONSUMIDO

localhost:30080

NodePort del Service de Kubernetes, no un proceso local.

CIERRE

Decisiones y limitaciones

Decisión	Motivo
Datos sintéticos	Drift controlado y repetible
RandomForest	El foco es la ingeniería
Modelo en la imagen	Imagen autosuficiente
Python 3.11 y versiones fijadas	Reproducibilidad
Bonferroni en data drift	Pocas falsas alarmas

Limitaciones

- Datos sintéticos, no reales
- Alias resuelto al exportar, no al arrancar el pod
- MLflow local sin autenticación

RESUMEN

Un modelo trazable, desplegado y vigilado

3
réplicas en Running

v1
del registry, con alias champion

Lote 7
alarma de concept drift

¿Preguntas?